

Image Segmentation

Nyamtulla Shaik



EECS 841, Spring 25

- ❑ Intro / CV Tasks
- ❑ Use Cases of Segmentations
- ❑ Segmentation Techniques
 - ❑ Traditional Methods
 - ❑ Deep Learning Models



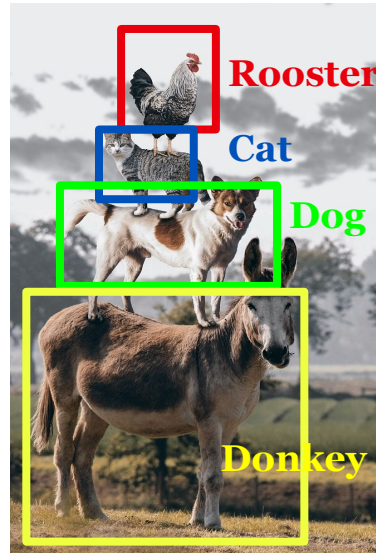
Computer Vision Tasks

Classification

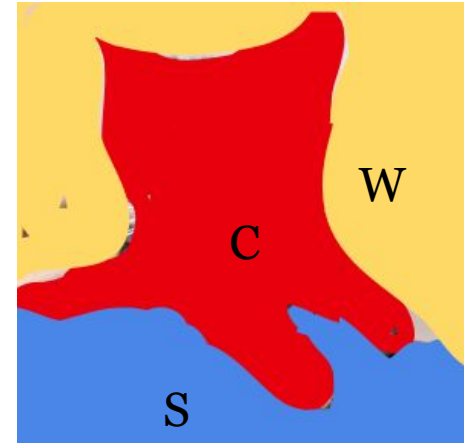


Cat

Object Detection



Segmentation



Wall, Cat, Sofa



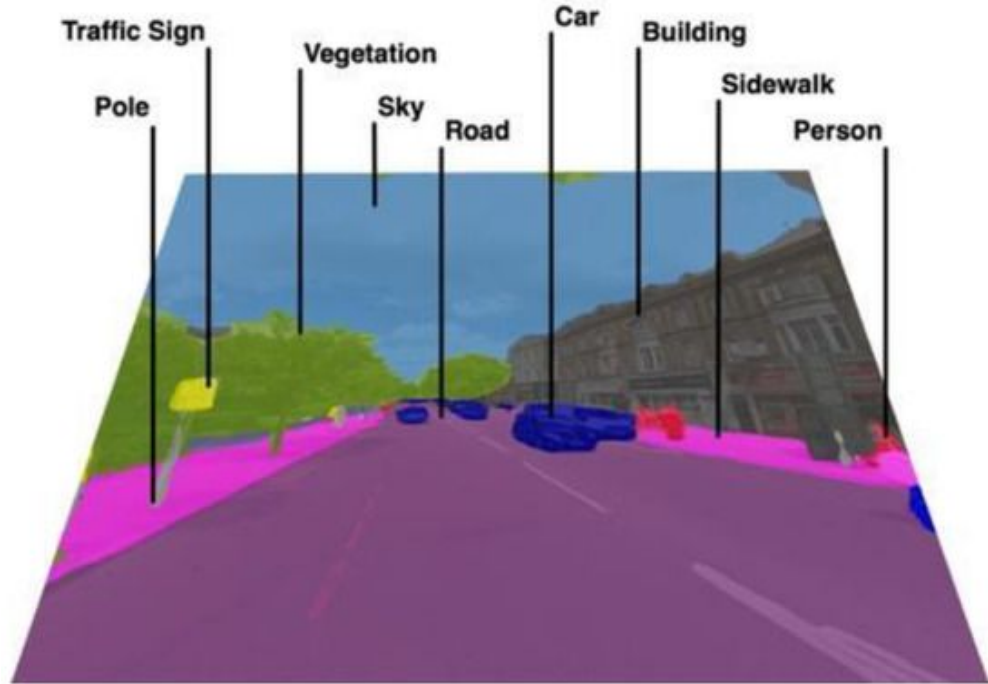
Segmentation

Image segmentation processes visual data at the pixel level, using various techniques to assign individual pixels as belonging to a specific class or instance.



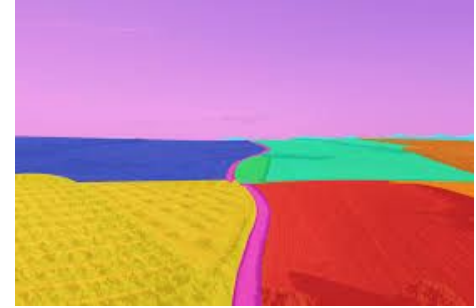
Segmentation

How can a car know where the road ends and sidewalk begins?

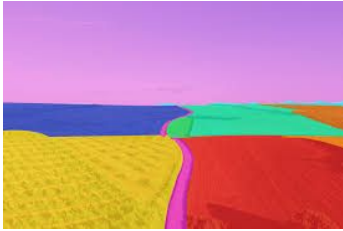


Use cases for image segmentation

- Autonomous vehicles
- Medical Imaging
- Satellite Imaging
- Agriculture
- Smart Cities
- Manufacturing



Use cases for image segmentation



“You’re not classifying the entire image anymore. You’re classifying every pixel”



Segmentation components

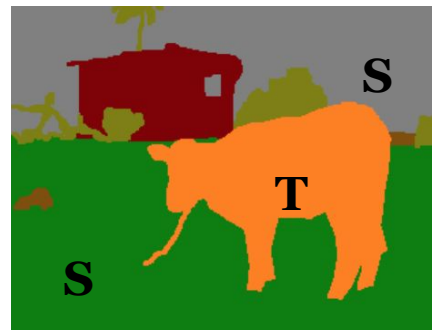
There are two categories of pixel segments:

Things

- Things corresponds to countable objects
- Things are classes of objects with characteristic shapes, like “car” or “tree” or “person”.
- They have relatively little variance in size from one instance to the next

Stuff

- Stuff refers to semantic classes that are amorphously shaped and highly variable in size, like “sky” or “water” or “grass”.
- Typically, stuff does not have clearly defined, countable individual instances.



Types of segmentations

The type of segmentation task depends on how a method treats the things and stuff classes

The three major types are:

Semantic



Instance



Panoptic



Semantic segmentation

- Semantic segmentation treats all the classes in an image as stuff
- It does not identify any objects
- It just assigns a semantic class to every pixel



(a) Source Image



(b) Semantic Segmantation

Images from ibm.com



Instance segmentation

- Instance segmentation isolates things from stuff.
- Instance segmentation describes the exact shape of each separate object instance.
- Consider, for example, how the two different models treat the parked cars in this image of a city street.



(a) Source Image



(b) Semantic Segmentation



(c) Instance Segmentation

Images from ibm.com



Panoptic segmentation

- Panoptic segmentation models both determine semantic classification of all pixels and differentiate each object instance in an image.
- It combines strategy of both semantic and Instance segmentation



(c) Instance Segmentation



(d) Panoptic Segmentation

Images from ibm.com



❏ Traditional Segmentations Methods



Segmentation methods, models and algorithms

Traditional / Classic image segmentation techniques determine annotations by analyzing inherent qualities of each pixel (called “heuristics”) like color and intensity.

- Fast and efficient for basic tasks like semantic segmentation.

Deep learning models employ complex neural networks for sophisticated pattern recognition.

- More precise and must for complex tasks like panoptic segmentation.



Traditional image segmentation techniques

- ❑ Clustering-based segmentation
- ❑ Region Growing -based segmentation
- ❑ Watersheds
- ❑ Edge detection
- ❑ Thresholding



Cluster-based segmentation

- Clustering algorithms make visual data into clusters of pixels with similar values.
- Each pixel is assigned to a cluster based on the nearest neighbour, which is most similar—centroid.
- Centroids are then relocated to the mean of each cluster and the process is repeated, relocating centroids with each iteration until clusters have stabilized.
- There are types of clustering approaches like **K-means** and **HCS**



Cluster-based segmentation : Evaluation Metric

There is an evaluation metric or clustering criteria to be followed

A **partition** of image is collection of sets $S_1, \dots, S_N$ such that

$$I = S_1 \cup S_2 \cup \dots \cup S_N \text{ (sets cover entire image)}$$

$$S_i \cap S_j = \emptyset, \text{ for all } i \neq j \text{ (sets do not overlap)}$$

A **predicate** $H(S_i)$ measures region homogeneity

$$H(R) = \begin{cases} \text{True, if pixels in region } R \text{ are similar} \\ \text{False, otherwise} \end{cases}$$

We want:

- Regions to be homogeneous (Property 1) $H(S_i) = \text{True for all } i$
- Adjacent regions to be different from each other (Property 2) $H(S_i \cup S_j) = \text{False for all adjacent } S_i, S_j$



Cluster-based segmentation :

Hierarchical Clustering Scheme (HCS)

- HCS is a popular approach to data clustering
- Segmentation can be viewed as a specific application of data Clustering
- HCS operates on a fully connected graph, containing a vertex for each data point and an edge between two data points.
- The weights of the edges are the distances between the data points as computed in some feature space

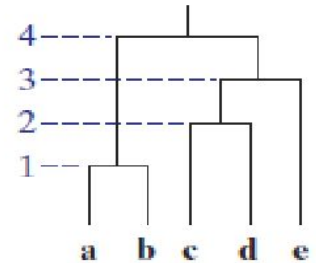
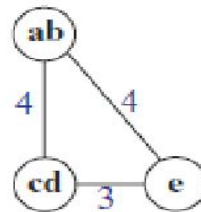
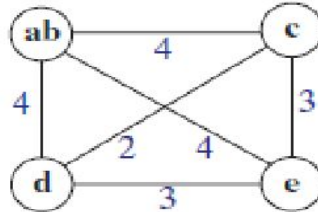
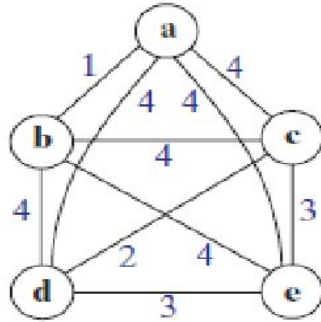


Cluster-based segmentation :

Hierarchical Clustering Scheme (HCS)

Clustering Process:

- Start with each data point as a separate cluster.
- Iteratively merge the two closest clusters (based on edge weights).
- No need to update distances (due to **ultrametric inequality**).



Dendrogram (Right):

- Visualizes the merge process as a tree structure.
- Horizontal axis: original data points.
- Vertical axis: distance at which clusters merged.
- Points merging at higher levels are less similar.



Region Growing -based segmentation

- Follows the clustering approach.
- These methods grow regions based on similarity in pixel values.
- Start with a seed pixel and grow the region by adding neighboring pixels with similar intensity or color.
- There are two types of region approaches **splitting** and **merging**



Region Growing: The two approaches

Splitting (Divisive clustering)

- Start with single region covering entire image
- repeat: split into homogeneous regions
- Even better: Repeat split cluster to yield two distant components

Property 2 is always True:

$H(S_i \cup S_j) = \text{False}$ for all adjacent

S_i, S_j

Goal is to satisfy Property 1:

$H(S_i) = \text{True}$ for all i

Merging (Agglomerative clustering)

- Start with each pixel as a separate region
- repeat: merge adjacent regions if union is homogeneous
- Even better: Repeat merge two closest clusters

Property 1 is always True:

$H(S_i) = \text{True}$ for all i

Goal is to satisfy Property 2:

$H(S_i \cup S_j) = \text{False}$ for all adjacent S_i, S_j in the final output

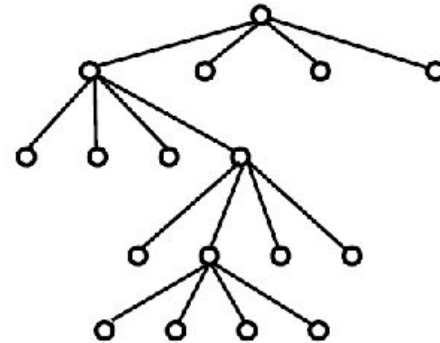
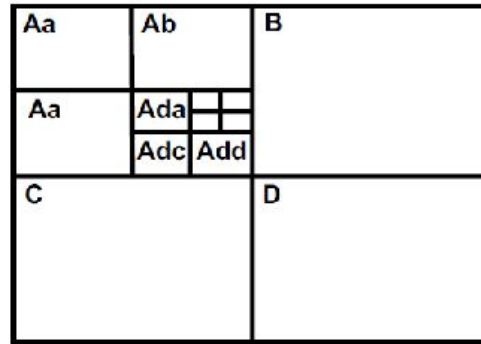


Region Growing-based segmentation : Splitting

- Start with entire image as a single region
- Repeat: Split any region that does not satisfy homogeneity criterion into subregions

$$H(R) = \begin{cases} \text{True, if pixels in region } R \text{ are similar} \\ \text{False, otherwise} \end{cases}$$

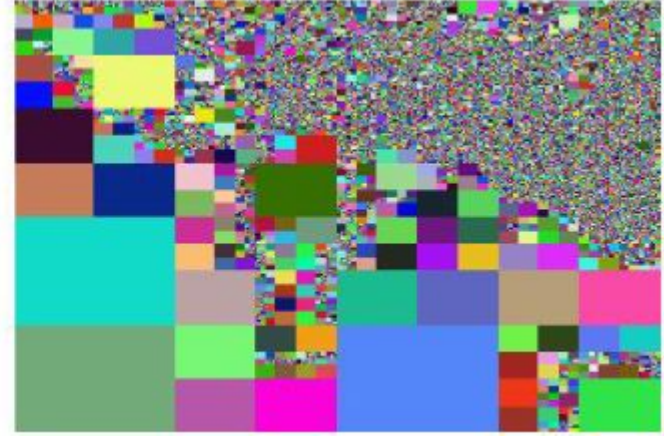
- Quadtree representation is convenient



Region Growing-based segmentation : Splitting



Original Image



After Split



Region Growing-based segmentation : Merging

- Start with a random seed pixel as a cluster
- Repeat:
 - Aggregate neighboring pixels that are similar to cluster model
 - Update cluster model with newly incorporated pixels
- This is generalized floodfill
- When cluster stops growing, begin with new seed pixel and continue
- An easy cluster model
 - Store mean and covariance of pixels in cluster
 - Use Mahalanobis distance to cluster
 - This leads to a natural threshold
 - Update mean and covariance efficiently.

Note : Since multiple regions are not grown simultaneously, threshold must be appropriate, or else early regions will dominate



Region Growing-based segmentation : Merging



Original Image



After Merging

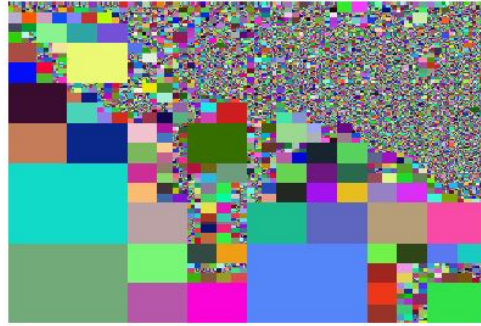


Region Growing-based segmentation: Split and Merge

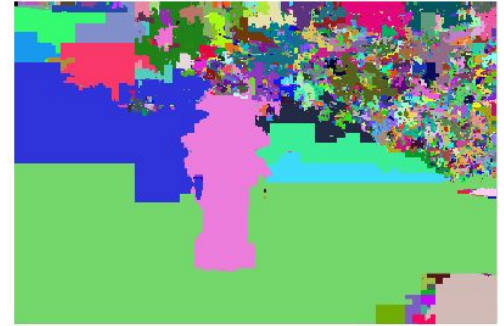
- Split-and-merge algorithm combines these two ideas
 - Split image into quadtree, where each region satisfies homogeneity criterion
 - Merge neighboring regions if their union satisfies criterion (like connected components)



Original Image



After Split

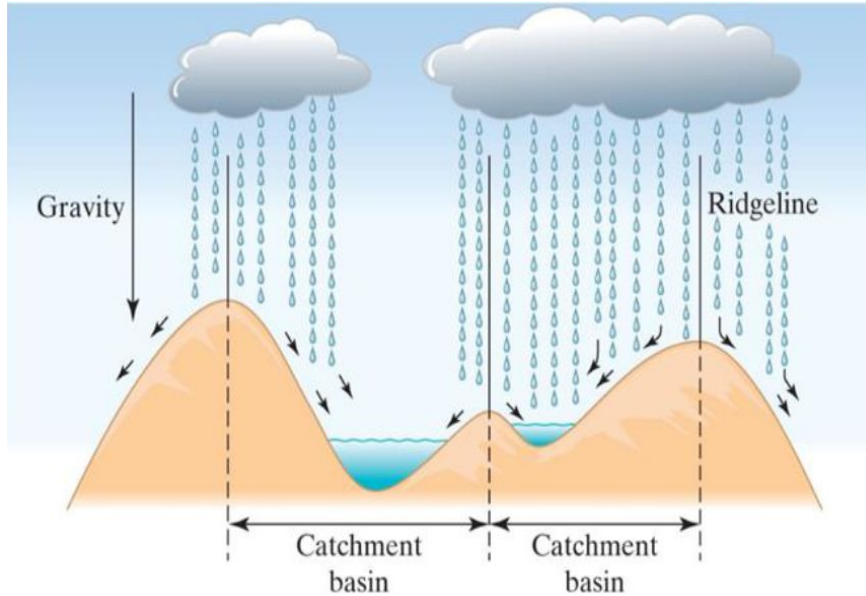


After Merging

S.L Horowitz and T.pavlidis, Picture Segmentation by a Tree Traversal Algorithm, 1976



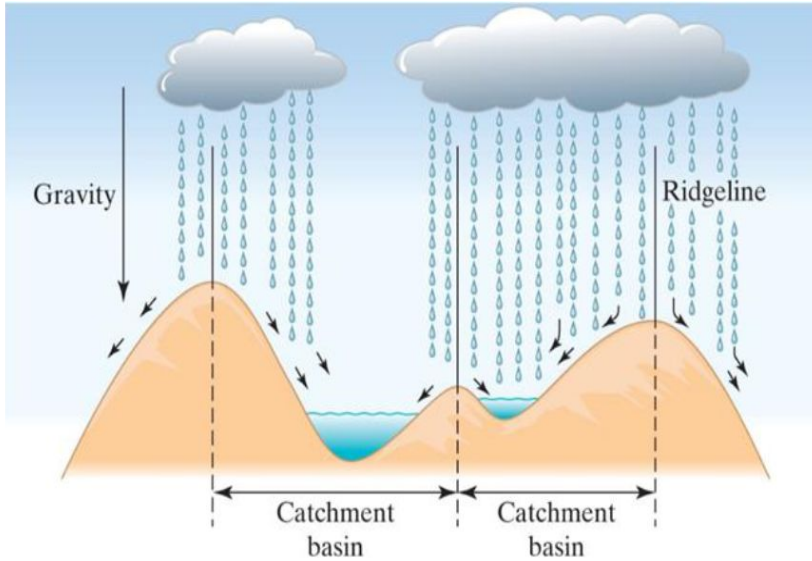
Watershed based segmentation



- Water fills the valleys (basins) from the lowest points upward.
- When two expanding regions come close, the algorithm places a boundary (dam) between them.
- These boundaries become the segmentation lines in the final image.
- The input image is treated like a topographic surface.
- Bright areas = high elevation (ridges), dark areas = low elevation (valleys).
- The **gradient magnitude image** is commonly used to define this surface.



Watershed based segmentation



The **gradient magnitude image** is commonly used to define this surface.

What is a Gradient Magnitude Image?

- The gradient of an image measures how quickly the pixel intensity is changing at each point.
- The gradient magnitude shows the strength of the edges in the image.
- It is computed using the derivatives in the x and y directions (usually with Sobel, Prewitt, or Scharr filters).



Watershed based segmentation

- Watershed segmentation combine ideas from region growing and edge detection.
- Segments an image into several “catchment basins “ or “regions”.
- The watershed approach to segmentation determines the segmented regions of the original image by computing the catchment basins of the terrain indicated by the segmentation function.
- The typical choice for the segmentation functions is the gradient magnitude of image.
- Objective: To find the watershed lines separating regions.



Watershed Methods *(divided into two major categories)*

1. Non-Marker-Based Watershed (a.k.a. Basic Immersion Watershed)

- Floods from every local minimum in the image.
- Requires only the gradient magnitude image (or the image itself).
- No guidance or constraints — every small intensity dip becomes a seed.
- Very prone to over-segmentation, especially with noisy data.

2. Marker-Based Watershed (Guided Segmentation)

- Starts flooding only from selected points (markers).
- Markers can be user-defined or generated via preprocessing (e.g., thresholding, distance transform).
- Helps control region growth and avoid over-segmentation.
- This is the improved and practical version used in most real-world applications.



Watershed Methods (Non-Marker Based)

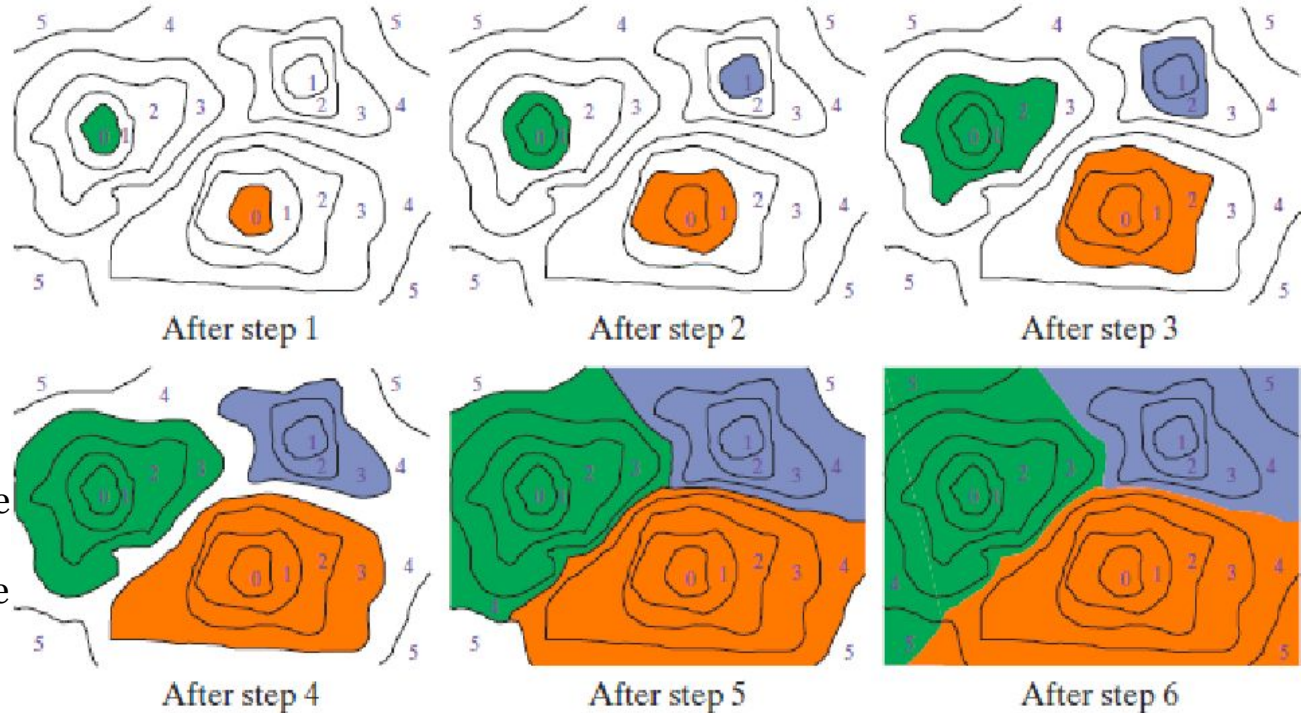
Water immersion (Vincent-Soille)

- Puncture hole at each local minimum, immerse in water.
- Grow level by level, starting with dark pixels.
- **Sorting step:** For efficiency, precompute for each gray level a list of pixels with that gray level (histogram with pointers)
- **Flooding step:**
 - Breadth-first search (floodfill) of level $g+1$ given flooding up to level g
 - For each pixel with value $g+1$, either assign to closest catchment basin or declare new.
 - catchment basin (geodesic influence zone).
 - repeat.

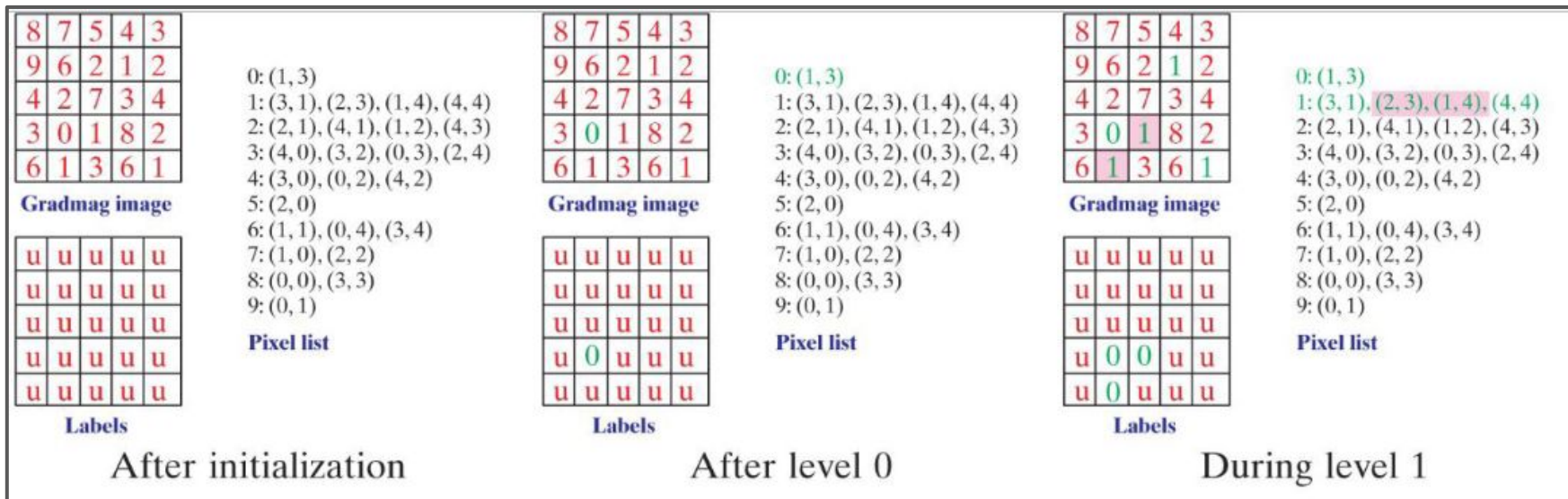


Watershed Methods: Water immersion (Vincent-Soille)

- Step-by-step results of immersion-based watershed segmentation on a segmentation function with 6 levels (0 through 5).
- The different colors indicate the unique labels of the three different regions.
- The contour lines of the segmentation function are shown, with numbers indicating the levels of the pixels.



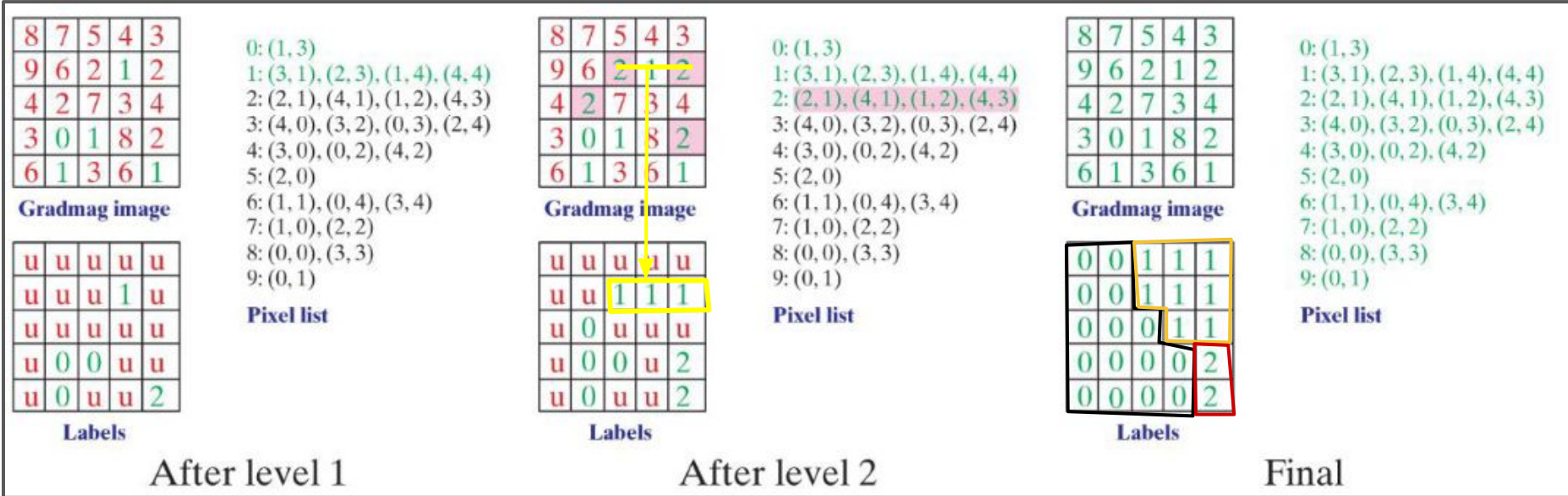
Watershed Methods (Non-Marker Based)



- An example of non-marker-based immersion watershed on a simple 5×5 image with 10 levels (0 through 9).
- Shown are the results after several steps of the algorithm, followed by the final result. (Note that ties are broken arbitrarily, so other solutions are possible.)
- At each step, pixel coordinates in the pixel list with a value less than or equal to the current value are colored, and pixels on the frontier are shaded.



Watershed Methods



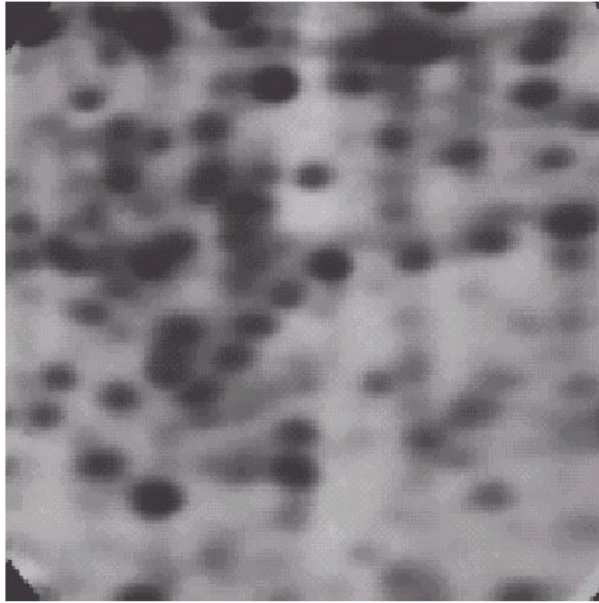
Non-Marker based Watershed Methods

Drawbacks

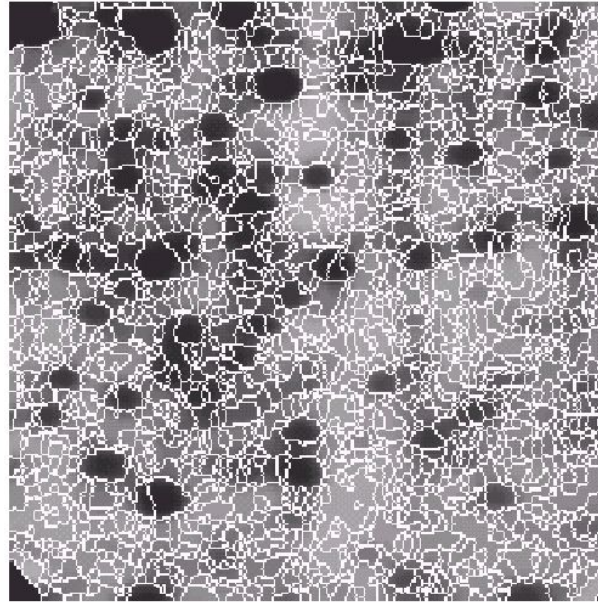
- Highly susceptible to noise in the segmentation function
- Any tiny wrinkle in the function will cause a local minimum, which will generate a unique label for a possibly small catchment basin containing the local minimum
- This leads to over-segmentation, and the errors from the standard algorithms are severe



Non-Marker based Watershed Methods : Over-segmentation problem



(a)



(b)

(a) Electrophoresis image.

(b) Result of applying the watershed segmentation algorithm to the gradient image. Oversegmentation is evident.

(Courtesy of Dr. S. Beucher, CMM/Ecole des Mines de Paris.)

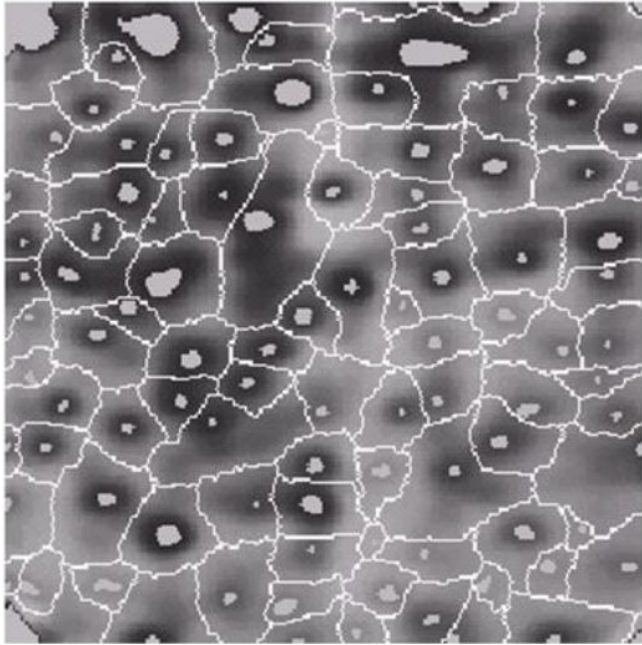


Watershed Methods (Marker based)

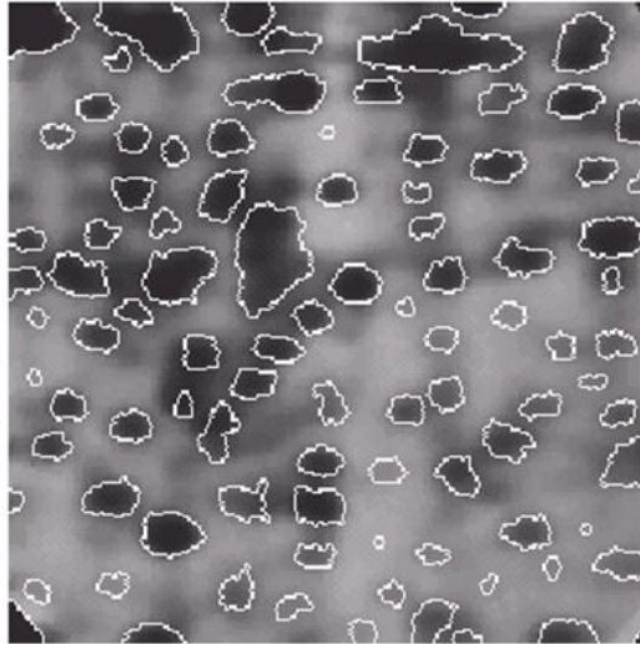
- The most common approach is to use marker-based watershed segmentation
- First use an independent procedure to generate a binary image where ON pixels indicates locations (markers) where the local minimum is significant



Watershed Methods : Over-segmentation



(a)



(b)

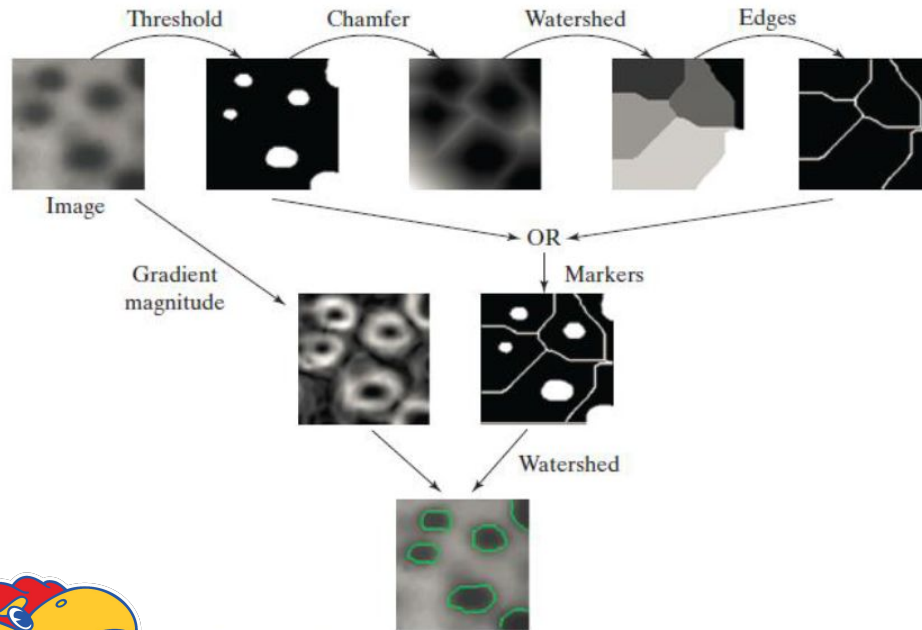
(a) Image showing internal markers (light gray regions) and external markers (watershed lines).
(b) Result of segmentation. Note the improvement over segmentation is evident.

(Courtesy of Dr. S. Beucher,
CMM/Ecole des Mines de Paris.)



Watershed Methods:

Marker-based watershed Algorithm

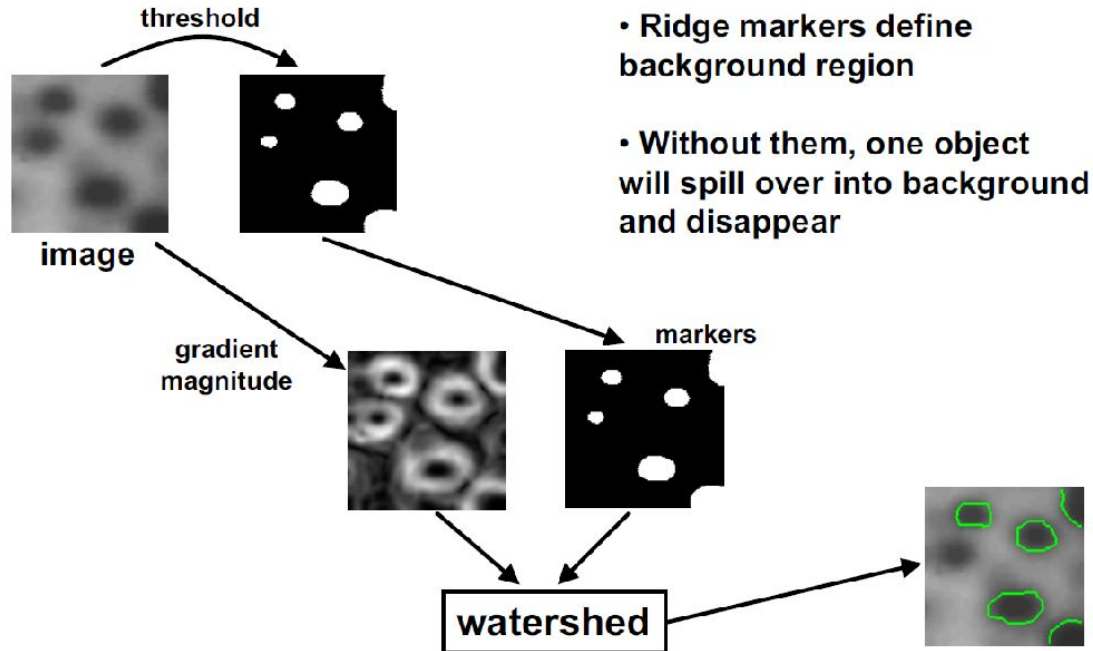


- Threshold the image.
- Compute chamfer distance.
- Run watershed to get lines between objects.
- Set these lines (skeletons) and blobs to zero in the gradient magnitude image.
- Only allow new basins where the value is zero.



Watershed Methods:

Marker-based watershed Algorithm



❏ Deep Learning based Methods



Key Challenges in Segmentation

- Scale variation
- Overlapping objects
- Boundary ambiguity
- Small object detection
- Real-time performance (speed vs accuracy tradeoff)



Why traditional methods fail?

- Can't handle complex scenes
- No learning of context or shape

Why deep learning succeeds?

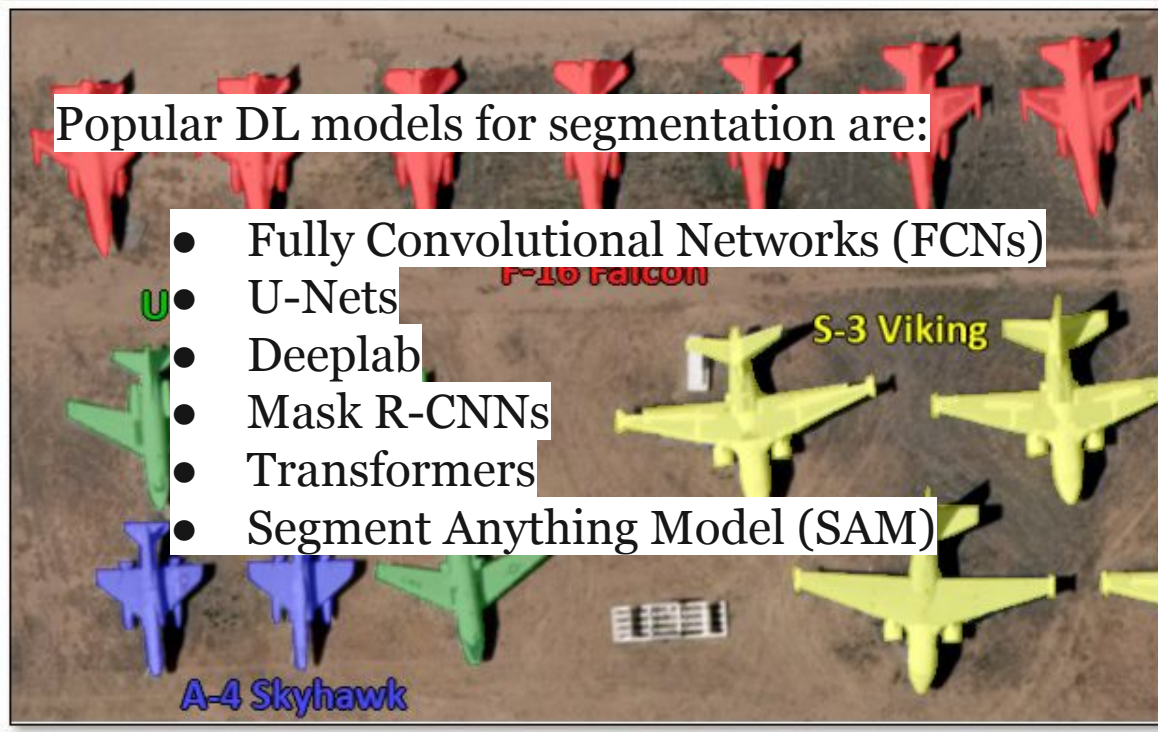
- Learns features automatically
- Captures both local (edges) and global (object context) information



Deep Learning Segmentation Techniques

Popular DL models for segmentation are:

- Fully Convolutional Networks (FCNs)
- U-Nets
- Deeplab
- Mask R-CNNs
- Transformers
- Segment Anything Model (SAM)



DL for Segmentation Evolution Timeline

Model	Year	Major Contribution
FCN	2015	End-to-end pixelwise prediction (first)
U-Net	2015	Encoder-decoder with skip connections
DeepLab v1–v3+	2015–2018	Atrous convolutions, ASPP, boundary refinement
Mask R-CNN	2017	Instance segmentation (mask prediction + detection)
Transformers	2021+	Global context via attention



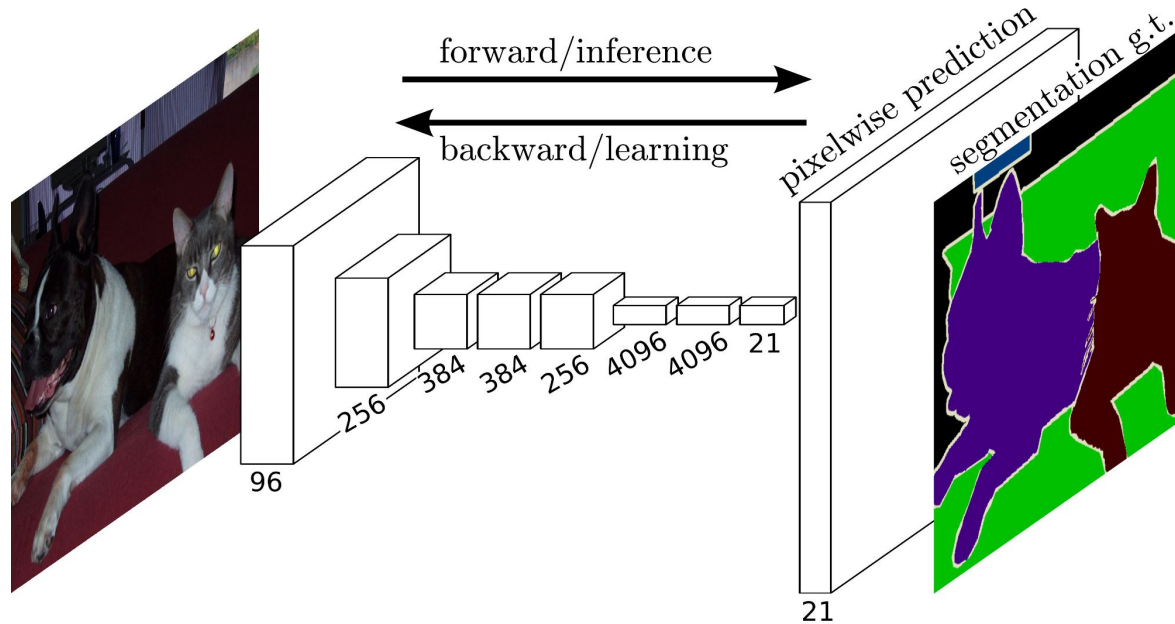
Fully Convolutional Networks (FCNs)

Why FCNs?

- Traditional CNNs (e.g., AlexNet, VGG) output a single label.
- Segmentation requires label for every pixel.
- Fully Convolutional Networks adapt CNNs for dense pixel-wise prediction.
- **Key Idea:** Remove fully connected layers, use convolution throughout.
- [Link to the Paper](#)



Fully Convolutional Networks (FCNs)



Fully Convolutional Networks (FCNs)

How Convolutions Solve the Problem!

- Convolutions preserve spatial structure.
- By using 1x1 convolutions at the final layer, each pixel has a class probability vector (softmax).
- No flattening.
- Output a 2D map where each location corresponds to a pixel.
- Allows variable input sizes and dense predictions.
- **Result:** Spatial correspondence maintained → segmentation possible.



Fully Convolutional Networks (FCNs)

Why Upsampling is Needed

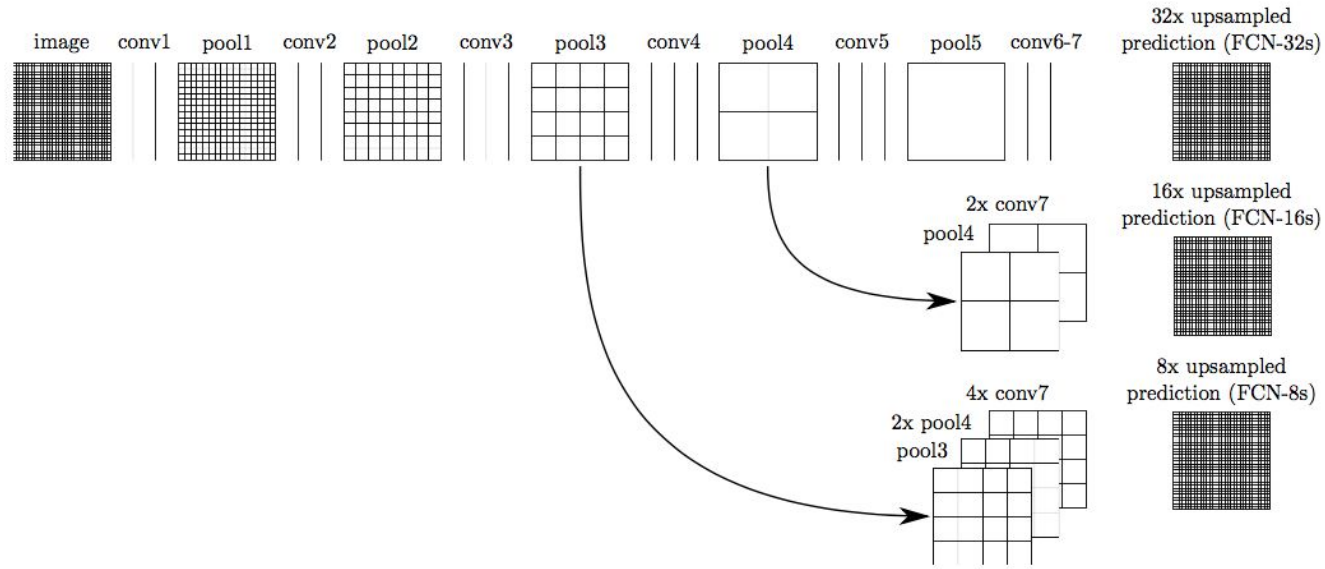
- Convolutions + pooling shrink feature maps (e.g., $224 \times 224 \rightarrow 28 \times 28$).
- We need output at original input resolution.

Solution:

- Use deconvolution (transpose convolution) layers to upsample.
- Alternatively, simple interpolation (bilinear) methods.



Fully Convolutional Networks (FCNs)



Fully Convolutional Networks (FCNs)

Skip connections

- To directly pass information from early layers (high-resolution, low-level features) to later layers during upsampling.
- They reconnect shallow feature maps to deep decoder layers.

Think of it like:

"Hey decoder, here's some fine details you missed earlier while focusing on the big picture — use them to draw better!"



Fully Convolutional Networks (FCNs)

Loss Function Used in FCN

- Pixelwise Cross-Entropy Loss:
- Predict class label for every pixel.
- Compare predicted map to ground truth segmentation mask.



Deep Learning Segmentation Techniques

Popular DL models for segmentation are:

- Fully Convolutional Networks (FCNs)
- **U-Nets**
- Deeplab
- Mask R-CNNs
- Transformers
- Segment Anything Model (SAM)



U-Nets

Why U-Net Was Proposed?

FCNs worked for segmentation, but had limitations:

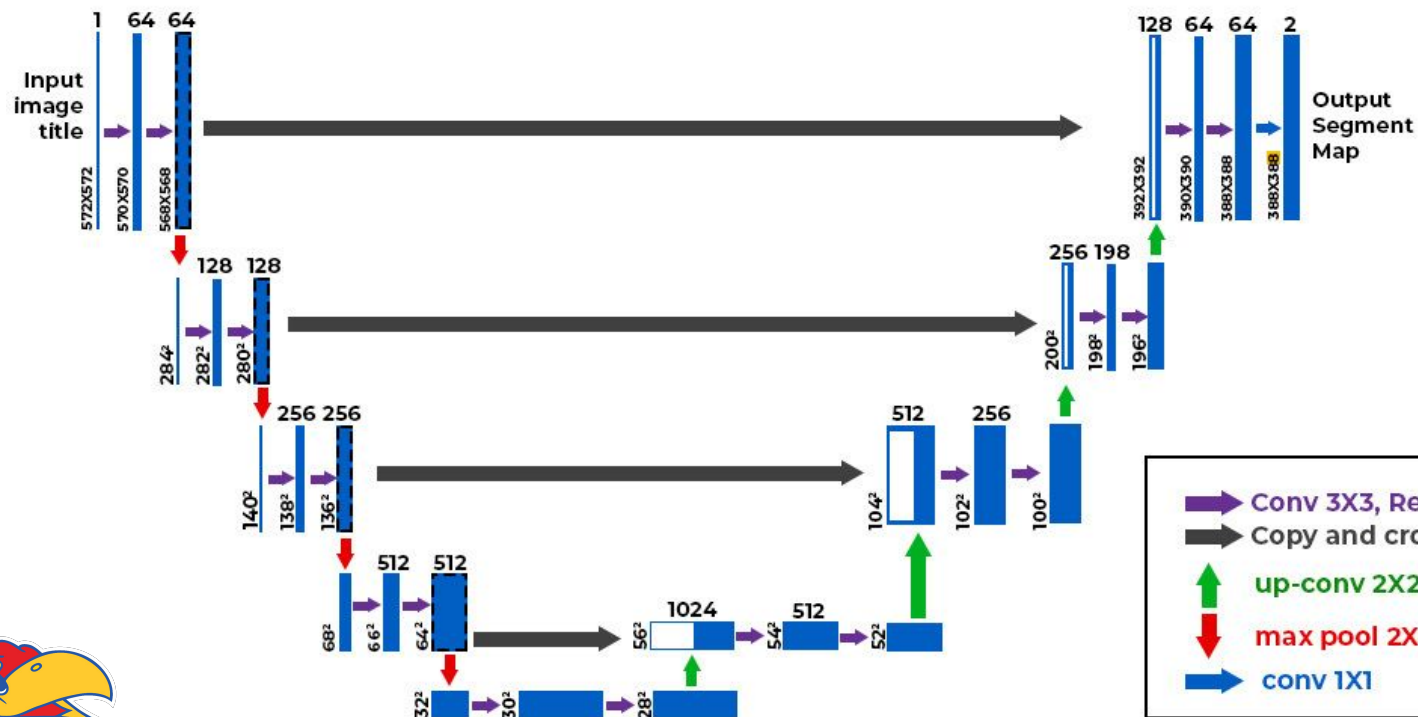
- Outputs still blurry.
- Difficult to precisely localize small objects.
- Medical images (e.g., cells, tumors) need very accurate boundaries.

U-Net was designed **specifically for biomedical image segmentation** — where pixel-level precision is critical.

[Link to the Paper](#)



U-Nets Architecture



U-Net Architecture Overview: U-Net is shaped like a big “U”

- **Left side:** Contracting path - **Encoder** (Downsampling)
 - a. Convolution (3×3) + ReLU
 - b. Max Pooling (2×2) to reduce size
- **Right Side** Expanding path - **Decoder** (Upsampling)
 - a. Transposed Convolutions (upsampling)
 - b. Convolution (3×3) + ReLU
 - c. Merge (concatenate) with corresponding encoder features (skip connections)
- **Final Layer**
 - a. 1×1 Convolution $\rightarrow$ Map to required number of classes (e.g., 2 for binary mask).



U-Nets

Skip Connections in U-Net?

- Directly copy and concatenate feature maps from encoder layers to decoder layers.
- Bring fine-grained details (edges, shapes) from early layers into the decoder.
- Helps produce sharp boundaries and accurate object segmentation.

Difference from FCN:

- FCN uses addition of feature maps.
- U-Net uses concatenation → richer feature representation.



U-Nets

Training U-Net

- Loss function:
 - **Pixelwise Cross-Entropy** for multi-class segmentation.
 - **Dice Loss** (especially for binary segmentation, class imbalance).
- Data augmentation heavily used:
 - Random shifts, rotations, elastic deformations.

Augmentation is critical because medical datasets are **tiny** compared to natural image datasets.



Deep Learning Segmentation Techniques

Popular DL models for segmentation are:

- Fully Convolutional Networks (FCNs)
- U-Nets
- **Deeplab**
- Mask R-CNNs
- Transformers
- Segment Anything Model (SAM)



Deep Lab Models

What Are DeepLab Models?

DeepLab is designed to capture large context while preserving spatial details.

DeepLab is **not one model** — it's a **series** of improvements:

- DeepLab v1: Atrous Convolution + CRF
- DeepLab v2: Atrous Spatial Pyramid Pooling (ASPP)
- DeepLab v3: Better ASPP
- DeepLab v3+: Encoder-Decoder refinement

[Link to the paper](#)



Deep Lab Models

What is Atrous (Dilated) Convolution?

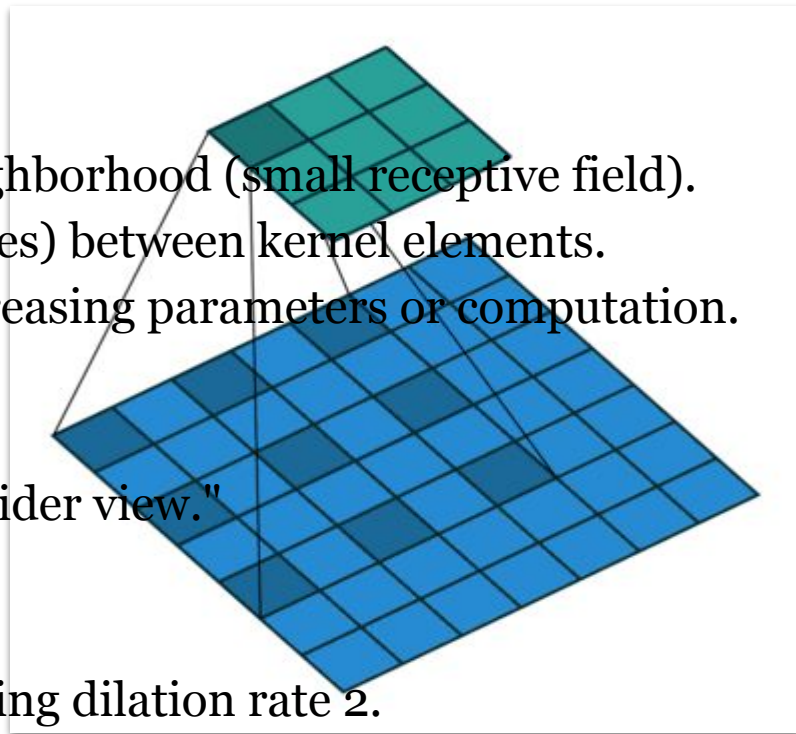
- **Normal convolution** captures local neighborhood (small receptive field).
- **Atrous convolution** inserts **holes** (zeroes) between kernel elements.
- Enlarges the **receptive field** without increasing parameters or computation.

Think of it like:

"Stretching the convolution filter to see a wider view."

Visual example:

3×3 conv $\rightarrow$ 7×7 effective field using dilation rate 2.



Deep Lab Models

DeepLab v1 Architecture

- Backbone CNN (e.g., VGG)
 - Replace last few layers with **atrous convolutions**.
 - Output coarse segmentation.
 - **DenseCRF post-processing:**
 - Dense Conditional Random Fields (CRFs) refine coarse segmentation.
 - Encourage pixels with similar appearance and spatial closeness to belong to the same label.
 - Works like a smart edge-aware smoothing.
- Solves blurry boundary problem without retraining network.



Deep Lab Models

DeepLab v2: Atrous Spatial Pyramid Pooling (ASPP)

- Even with dilation, different objects are at **different scales**.
- Use **multiple parallel atrous convolutions** with **different dilation rates**.

ASPP Module:

- Captures multi-scale context:
 - Small dilation → fine details
 - Large dilation → large objects



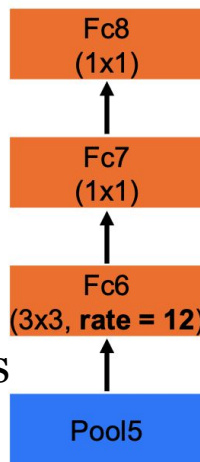
Deep Lab Models

ASPP Explained

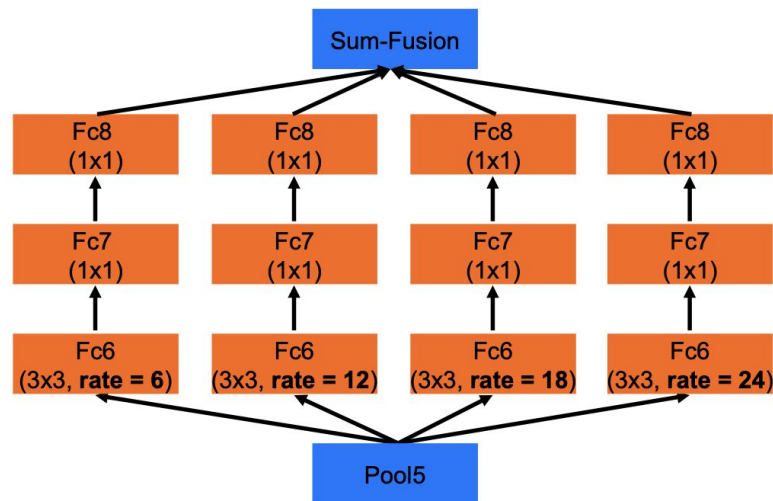
Imagine several parallel convolutions:

- 3×3 kernel, dilation=6
- 3×3 kernel, dilation=12
- 3×3 kernel, dilation=18
- Output is 1×1 convolution

Then **concatenate** the outputs together.



(a) DeepLab-LargeFOV



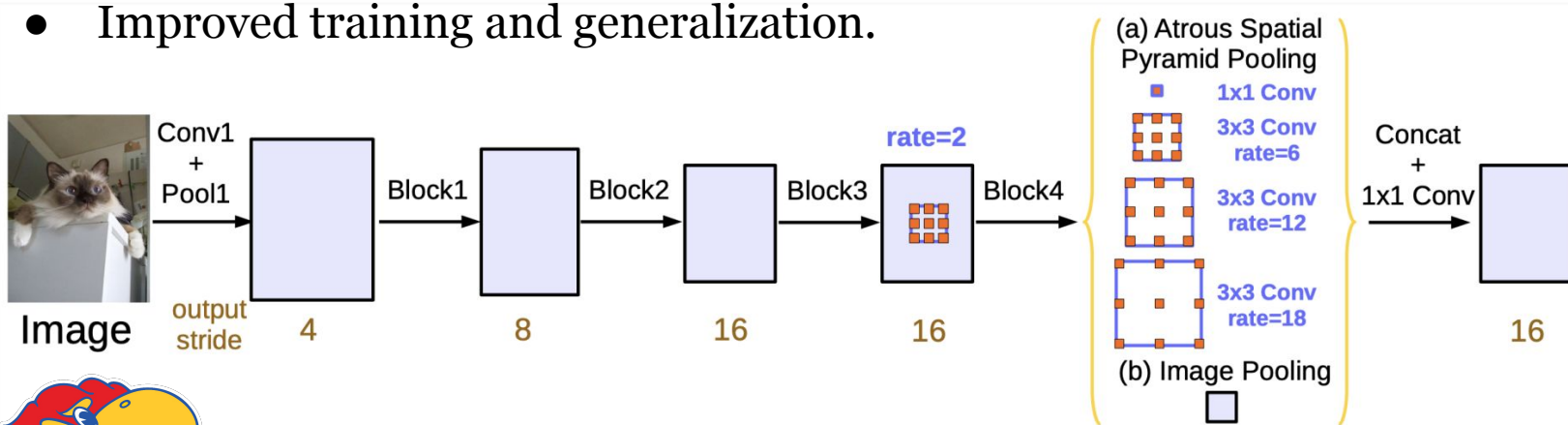
(b) DeepLab-ASPP



Deep Lab Models

DeepLab v3: More Powerful ASPP

- Add **Batch Normalization** after each atrous conv.
- Add **global average pooling branch** (capture truly global context).
- Improved training and generalization.



Deep Lab Models

DeepLab v3+: Adding Encoder-Decoder [\(paper\)](#)

Even ASPP outputs are still coarse at object boundaries.

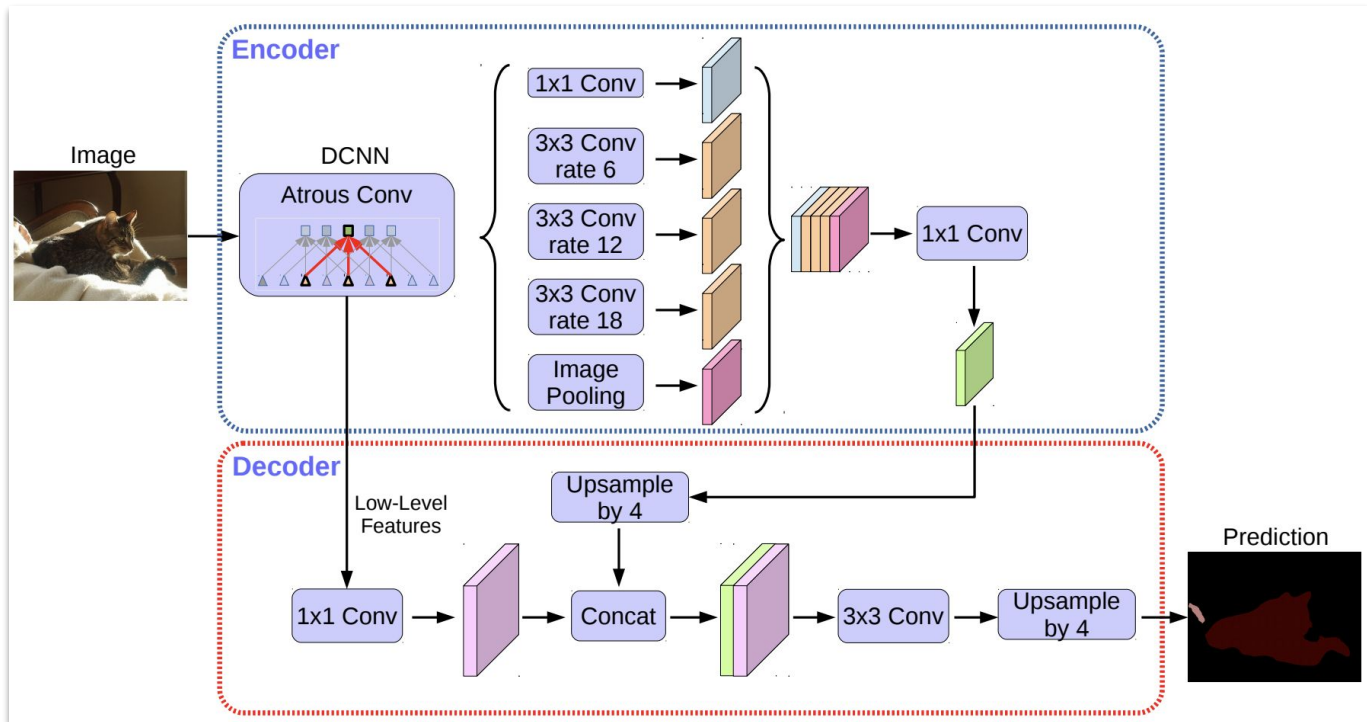
DeepLab v3+ architecture:

- Encoder = Deep network with ASPP.
- Decoder = Lightweight network that **refines segmentation map**, especially around edges.

Very sharp boundaries, almost perfect segmentations.



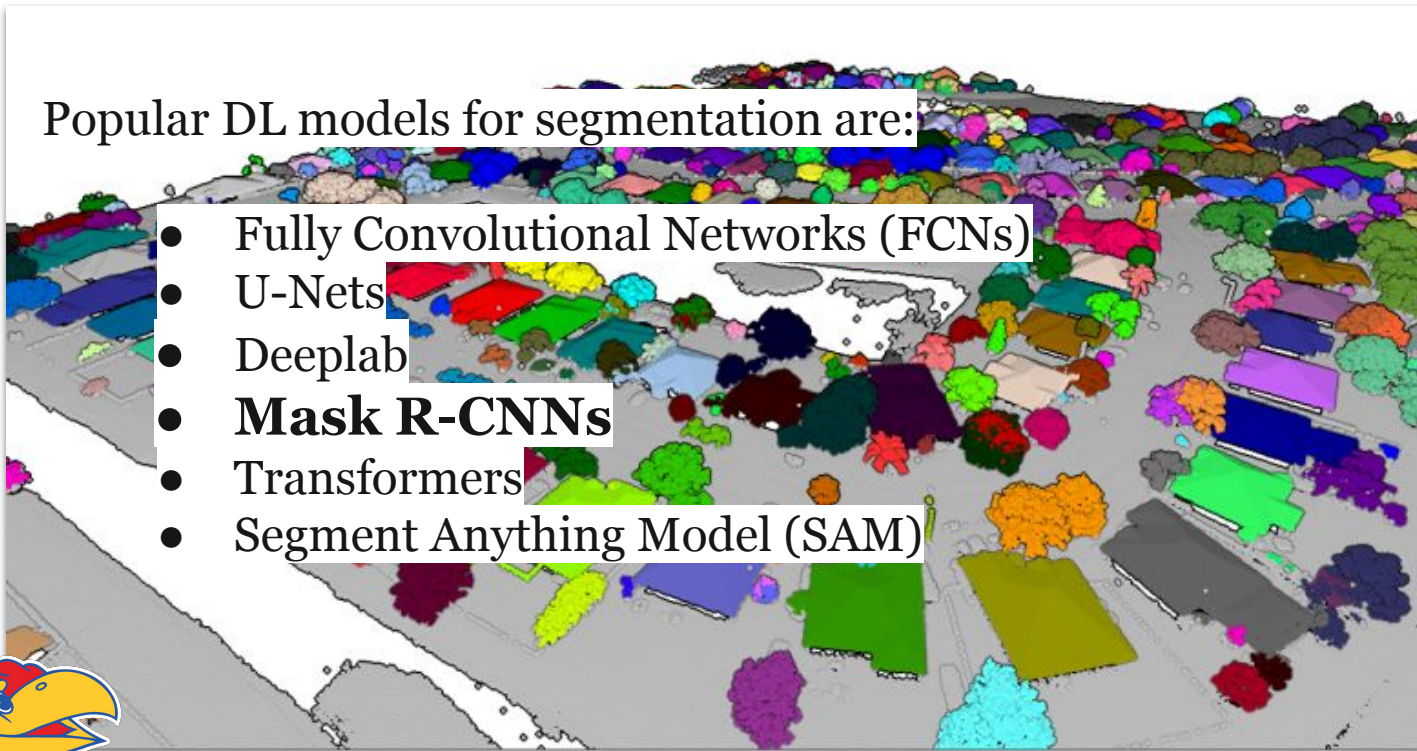
Deep Lab Models



Deep Learning Segmentation Techniques

Popular DL models for segmentation are:

- Fully Convolutional Networks (FCNs)
- U-Nets
- Deeplab
- **Mask R-CNNs**
- Transformers
- Segment Anything Model (SAM)



Mask R-CNN

Why Mask R-CNN Was Proposed?

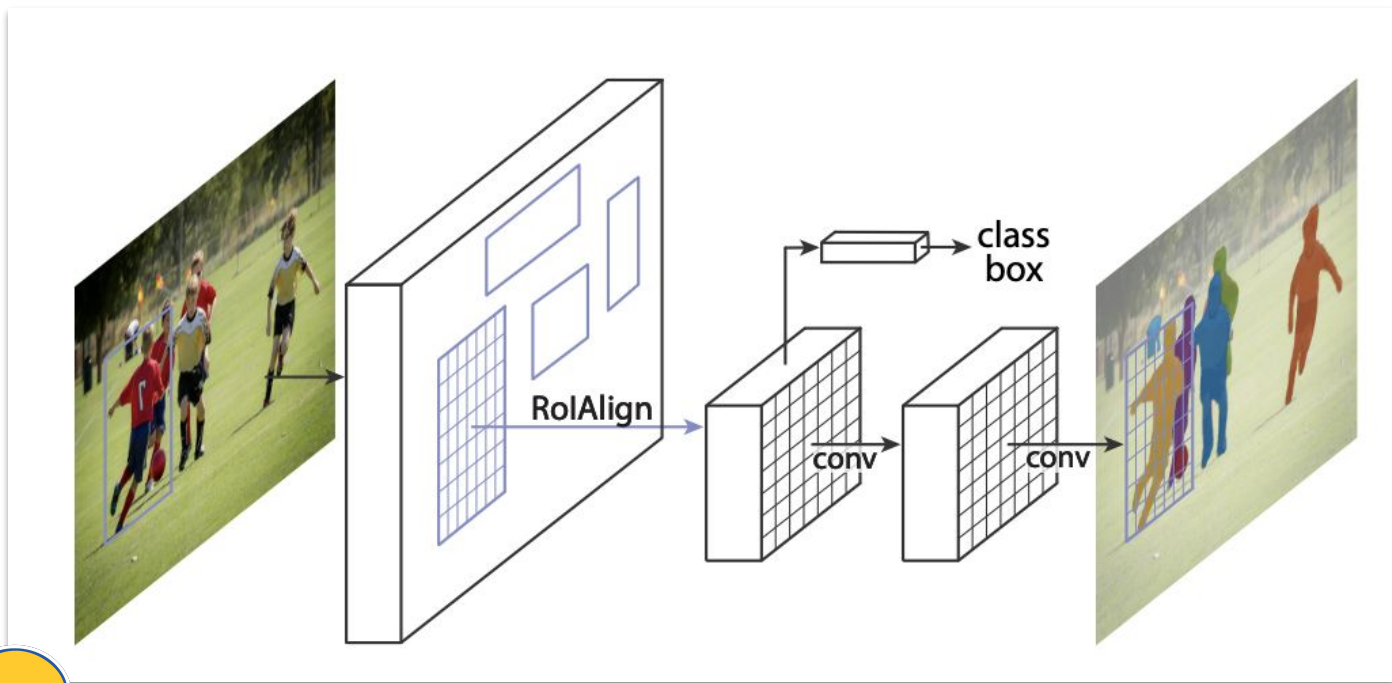
- Semantic segmentation (FCN, U-Net, DeepLab) **labels each pixel**.
- But what if there are **multiple objects** of the same class (3 cats)?
- We need **Instance Segmentation**:
 - Separate **individual object instances** and label them separately.

Mask R-CNN was designed to **detect objects and generate a mask for each object**.

[Link to the paper](https://arxiv.org/pdf/1703.06870) - <https://arxiv.org/pdf/1703.06870>



Mask R-CNN Architecture Overview



Mask R-CNN

Mask R-CNN Architecture Overview

1. **Backbone** (ResNet + FPN) extracts features.
2. **Region Proposal Network (RPN)** proposes candidate object regions(bounding boxes).
3. **ROI (Region of Interest) Align** extracts exact feature maps for each proposal.
4. Three parallel heads:
 - **Classification Head:** Predict object class.
 - **Bounding Box Head:** Refine box coordinates.
 - **Mask Head:** Predict binary mask for object inside box.



Mask R-CNN

How Mask Head Works

- For each Region of Interest (RoI):
 - Predict a **small spatial binary mask** (e.g., 28×28 resolution).
- Predicts **one binary mask per class**.
- Only the **mask corresponding to the ground truth class** is used in loss.

Mask prediction is **pixel-wise binary classification** inside the object bounding box.



Mask R-CNN

Applications of Mask R-CNN

- Self-driving cars (detecting individual pedestrians, cars)
- Medical imaging (tumor segmentation instance-wise)
- Video object segmentation
- AR/VR object interaction
- Image editing (cutting out objects)



Deep Learning Segmentation Techniques

Popular DL models for segmentation are:

- Fully Convolutional Networks (FCNs)
- U-Nets
- Deeplab
- Mask R-CNNs
- **Transformers**
- Segment Anything Model (SAM)

*“we are moving into **the latest generation** of segmentation models!”*



Transformers

Why Move Beyond CNNs for Segmentation?

- CNNs work **locally** (small receptive field initially).
- CNNs struggle with **long-range dependencies**:
 - Knowing relationships between distant pixels (e.g., head and feet of a person).

Transformers bring **global attention**:

- Any pixel can directly attend to any other pixel.
- Better **context aggregation**.



Transformers

Basic Transformer Idea (for Vision)

- Originally designed for **sequences** (NLP).
- Self-attention computes relations between all pairs of tokens.

For images:

- **Split image into patches** (e.g., 16×16 patches).
- Flatten patches $\rightarrow$ **patch tokens**.
- Feed tokens into standard transformer encoder.

Result: Image is treated as a **sequence of patches**.

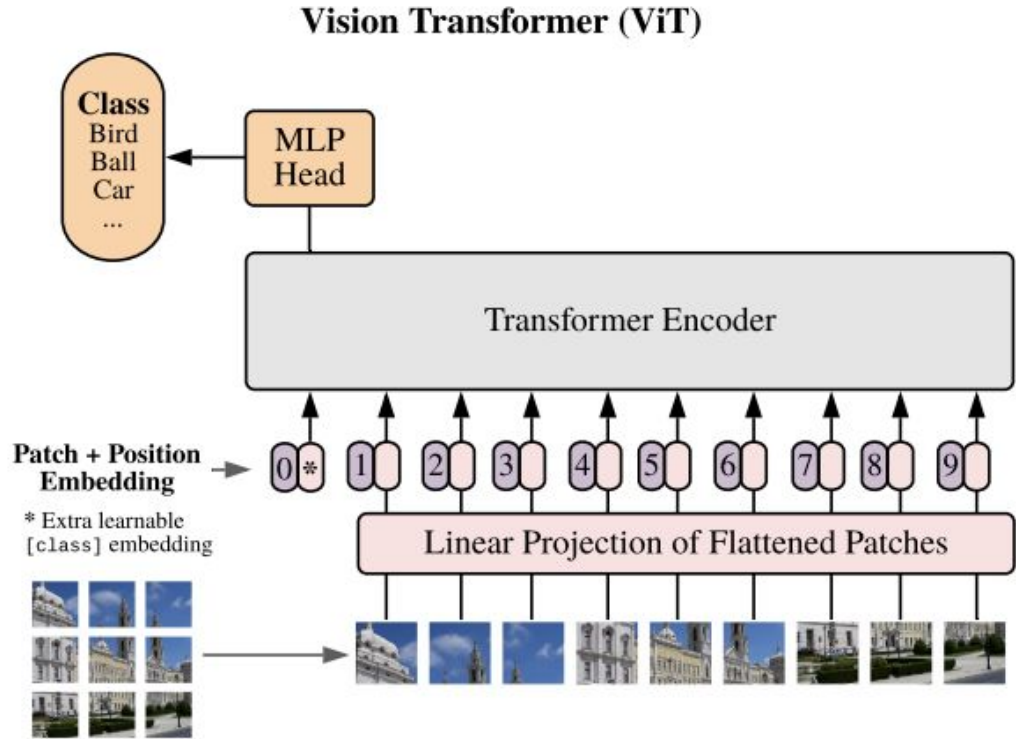


Transformer Based Models for Segmentation

Model	Key Idea
ViT (Vision Transformer)	Classification from patch tokens
SETR (SEgmentation TRansformer)	Pure transformer encoder-decoder for pixel labeling
Segmenter	Predict mask tokens for segmentation
SegFormer	Lightweight and hierarchical transformer for segmentation
Mask2Former	Universal mask prediction for all segmentation tasks



Vision Transformer (ViT)



Vision Transformer (ViT)

- Input image → split into patches.
- Each patch → flatten + linear embed + positional encoding.
- Sequence → fed to transformer encoder (multi-head self-attention).

Very strong for classification. But **vanilla ViT struggles** with dense pixel-wise predictions (no locality preserved).

[Link to the paper](https://arxiv.org/pdf/2010.11929) - <https://arxiv.org/pdf/2010.11929>



SETR (SEgmentation TRansformer)

- **SETR** (SEgmentation TRansformer) was one of the first.
- Pure transformer encoder to extract features.
- Simple upsampling (deconvolution) decoder to map features back to segmentation.

Problem:

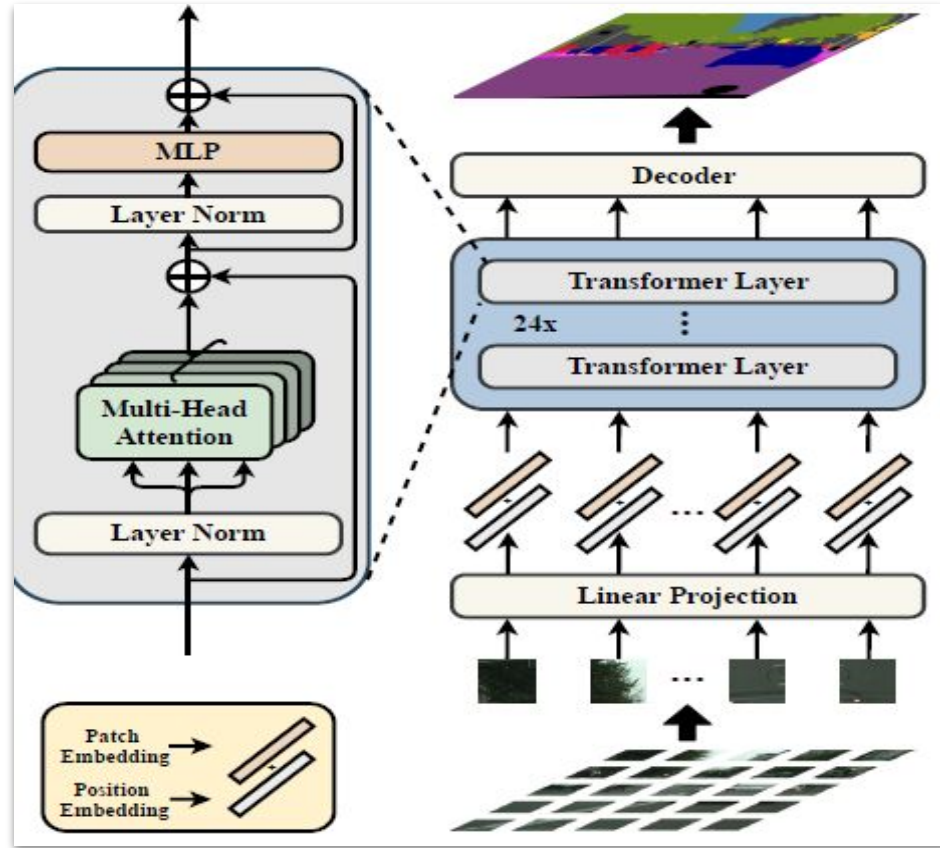
- Heavy computation (quadratic complexity).
- Poor at preserving local structure (small objects, boundaries).

[Link to the paper](https://arxiv.org/pdf/2012.15840) - <https://arxiv.org/pdf/2012.15840>



SETR

(SEgmentation
TRansformer)



SegFormer: Efficient Transformer for Segmentation

- **SegFormer** addresses SETR problems:
 - Lightweight hierarchical transformer (multi-scale features).
 - No position embeddings — model learns positional information implicitly.
 - Simple MLP decoder — no heavy upsampling.

Advantages:

- Extremely fast.
- Great for real-time segmentation tasks (autonomous driving, robotics).



[Link to the paper](https://arxiv.org/pdf/2105.15203) - <https://arxiv.org/pdf/2105.15203>

Mask2Former: Universal Segmentation Framework

Before this semantic, instance, and panoptic segmentation needed different architectures.

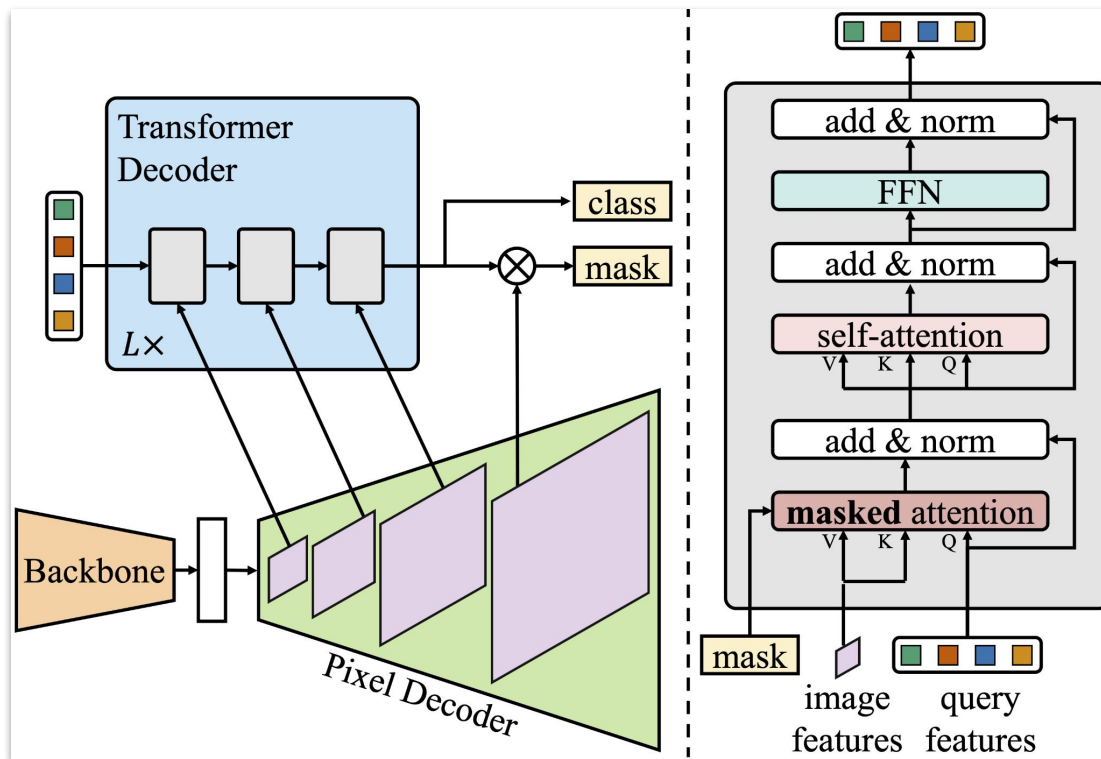
- **Mask2Former** predicts **set of masks** instead of pixel labels.
- Mask prediction with attention mechanism.
- Pixel decoder extracts high-res features.
- Transformer decoder predicts mask embeddings.
- Masks + classes predicted simultaneously.

Same architecture handles semantic, instance, and panoptic segmentation.

[Link to the paper](https://arxiv.org/pdf/2112.01527) - <https://arxiv.org/pdf/2112.01527>



Mask2Former: Universal Segmentation Framework



Transformer vs CNN in Segmentation

Aspect	CNN	Transformer
Local/Global	Local context first, global later	Global context from start
Receptive Field Growth	Gradual	Immediate
Segmentation Sharpness	Good with techniques	Often sharper naturally
Scalability	Efficient at small scale	Heavy computation, improving



When to Prefer Which Model?

Scenario	Recommended Model
Simple binary segmentation	U-Net
Medical imaging	U-Net or DeepLab
Urban scene semantic segmentation	DeepLab v3+, SegFormer
Instance segmentation (individual objects)	Mask R-CNN
Real-time segmentation (robotics, drones)	SegFormer
Very large, complex scenes	Mask2Former



Segment Anything Model (SAM)

Foundation Models for Segmentation

Segment Anything Model (SAM) — Meta AI, 2023

<https://segment-anything.com/>

[Paper](#)



Segment Anything Model (SAM)

Traditional models (FCN, U-Net, DeepLab, Mask R-CNN, Transformers):

- Trained for **specific classes** (e.g., dog, cat, road).
- Need retraining for new tasks.

SAM is different:

- Trained on **1 billion masks** across **11 million images**.
- **Promptable**: can segment **any object** with a **click, box, or mask**.
- **Zero-shot generalization**: no retraining needed for new classes.

SAM turns segmentation into a **general interactive task**, like GPT did for text.



How SAM Works (High-Level Architecture)

- **Image Encoder:**
 - Powerful Vision Transformer (ViT-Huge) extracts dense feature maps.
- **Prompt Encoder:**
 - Encodes user input (points, boxes, masks).
- **Mask Decoder:**
 - Combines image features + prompt features → predicts segmentation masks dynamically.

Works with **multiple prompts** and **outputs multiple masks** in parallel.



Key Innovations in SAM

Feature	Innovation
Foundation Model	Generalizes across tasks without retraining
Prompt-based segmentation	Dynamic interaction with user
Massive training data	1B masks, 11M images
Zero-shot performance	Segments unseen objects easily



Future of DL based Segmentation

Efficient transformers
+
Universal segmentation frameworks !!



Thank You

